COMP2006, 2025/2026, Coursework Requirements: GUI Framework

Contents

Overview

This coursework has two parts – an initial 'easier' part to get you working on this earlier, and the final part which is more complex and lets you innovate more. Your part 2 coursework can be completely different and separate to your part 1 – you do not need to build on the part 1 submission to make part 2, although doing so may save some work.

Part 1 aims to introduce you to a C++ framework for building GUI programs. You should be able to get full marks on that part if you work through the demos and spend time doing the coursework. **Part 2** aims to give you the freedom to do more with the framework and potentially create something impressive. The aim is that your total mark for part 1 and part 2 should be representative of your ability with C++ (or the effort you put into the coursework).

The coursework framework is a simplified cut-down framework covering just the essentials and aims to simplify the process of software development for you. It may seem large to you but in fact is very small compared with the sizes and complexities of class libraries you would usually be working with in C++. You will need to understand some C++ and OO concepts to be able to complete the coursework but will (deliberately) not need to understand many complex features initially, so that the coursework can be released earlier in the semester.

There are two demo tutorials (labelled Demo A and Demo B) which you should work through to help you to understand the basics. Please complete these to understand how to do part 1. There are also some examples in the coursework code provided, to illustrate how you can do some things. Your submissions must be different enough from the demo tutorials and supplied examples that it is clear to the markers that you do understand the concepts rather than just copying them.

Part 1 is designed to be easy to complete. (You should be able to check the marking criteria in advance and know what mark to expect – hopefully full marks for each of you.) A zip file of your project (including all source, project and resource files), as well as a demo video (**up to 3 minutes**) showing how you meet the marking criteria MUST be uploaded to Moodle.

Part 2 is more advanced and is designed to be harder to complete. Part 2 is designed to be hard in some parts! You may not be able to do all parts. Please *do what you can and make sure it compiles and runs etc*. Submission of documentation, code and the demo video (**up to 6 minutes**) will be via Moodle.

Failing to upload your project, documentation and the demo video will count as a failed submission and result in a mark of 0.

You may complete and demo the coursework on Windows, Linux or Mac. As long as you can submit your code and do us a demo, we don't mind what operating system you use.

Getting started:

1. **Download the zip file of the coursework framework. Unzip it and open it in Visual Studio** – on your own computer or the windows virtual desktop. Compile and run the initial framework to ensure that it compiles and runs with no problems on your computer. Read the getting started document if you are stuck. You can also use the lab PCs or the windows virtual desktop.

   (Mac, Linux) Optionally: If you prefer then you can follow the instructions provided on Moodle to build on Mac or Linux. We are assuming that, if you wish to do this, it is because you are the expert on Mac or Linux so will be able to resolve any issues. This should ensure that those who are more comfortable on a different operating system are able to use it, but please be aware that we may not be able to support issues on these platforms.

   Whatever you run this on, you will need to be able to demonstrate your work for marking as well as to submit your source code files to Moodle. (It should be possible for you to demo on any platform you wish and/or demo on the windows virtual desktop, if necessary, by copying files back into the visual studio project.)

2. **Do the coursework tutorial demo A** and ensure that you understand about the basic features of the BaseEngine class and drawing the background, including the tile manager and handling input.

3. **Do the coursework tutorial B** and ensure that you understand about the basic features of simple displayable objects, moving them around and displaying them.

4. **Skim through the Frequently Asked Questions document to ensure that you know what is in it.** This document has developed over the years to address questions which I often get asked. It will also give you clues to things you may not know of may not think of.

5. **Start on the requirements for part 1**, using what you learned from demo tutorials A and B. Coursework part 1 should not be too bad using the walkthroughs for demo A and demo B and the samples. Try changing which object type is created in the mainfunction.cpp file (change which line is commented out to change which demo to run) and look at the provided demos.
   You may want to consider the 'SimpleDemo' demo, for things like string displaying, and potentially either the BouncingBall or Maze demos if you can't work out things like the TileManager. Work on only the simple demos initially! You will not need the more advanced demos until you get to part 2.

6. **Once you have completed part 1, look at the part 2 requirements and decide on how you will develop a program to be able to do as many of these as you wish, considering what you learned from part 1.** You should think about which requirements you will attempt at the start rather than trying to consider this later because some programs will make them more complex to achieve than others. Note: in some cases, you may find it impractical to integrate some requirements into your main program. In that case, don't forget that you could use a fancy 'intro' or 'ending' screen (state) to demo things that may not fit into your main program (e.g. TileManager in past years has not always fitted for some students, and has been used on an intro screen). Think outside the box and **remember that your aim is to get the best mark, not be exact in matching some existing program idea or to make the best game ever.**

General requirements (apply to both parts):

You MUST meet the following general requirements for both part A and part B:

1. **All the code that you submit must be either your own work or copied from the supplied demos.** Where you use code from demos it will not get your marks unless you have modified it enough to show your own understanding.
   You may NOT use work from anyone else.
   You may NOT work with other people to develop your coursework.
   You may NOT have someone else write any or all the code in your submission.
   You will be required to submit a simple documentation sheet making clear that you understand this.
2. **Create a program which runs without crashing or errors and meets the functional requirements** listed below and on the following page. Look at the Hall of Fame page to see some examples of previous work and decide on a basic idea for your program. You don't need to implement that much for part 1, but part 2 will involve finishing it.
3. Avoid compilation warnings in your code. Usually these mean that there is a potential issue with your program. Try to avoid compilation warnings as any warnings may make us investigate whether they mean that there are C++ issues with your program.
4. **Your program features which are assessed must be different from all the demos and from exercises A and B.** i.e. do not copy demo code or the lab exercises code too closely. You will not get marks for copying code but would get some marks for your own modifications which show your understanding of C++ if you did *similar* things to the demos. In other words, change the code at least enough to show that you understand what it is doing and haven't just blindly copied it.
5. **You must not alter existing files in the framework** without prior agreement with from the module convenor (which would require a good reason) – the aim of the coursework is to work with an existing framework not to improve it (even though there are many possible improvements).
   Note: subclassing existing classes and changing behaviour in that way is permitted, and is encouraged, as you don't need to change the existing files to do that.
   I tried to make the classes flexible enough to allow behaviour to be modifiable by subclasses relatively easily.
6. The aim of this coursework is to investigate your ability to understand existing C++ code and to extend it using sub-classing, rather than to use some alternative libraries:
   - **Your program MUST use the supplied framework in the way in which it is intended**. i.e. the way in which demo tutorials A and B use it to draw the background and moving objects is the way that you should use it.
   - **You must not alter the supplied framework files.** See comments above for point 5.
   - **You should not use any external libraries other than standard C++ class libraries**, and libraries which are already used by the framework code, unless a requirement specifically says you may do so. There are many useful libraries which you could use to improve your programs, but the point of this exercise is to show that you can understand and use the library I supply.
7. **There are 10 functional requirements in part 1**, worth 2% of the module mark each, for a total of 20% of the module mark for this part of the coursework.
   When you have completed the coursework part 1, **submit a zip of your source code and a demo video to Moodle:**
   - **Clean your program** (use the 'Clean Solution' option under the build menu – to delete the temporary files.

- **Delete the hidden .vs folder in the root of your project directory**. This can become huge but is not needed and will make your zip file unnecessarily big.
- **Zip up the rest of your project directory** and submit it to 'part 1 coursework submission' on Moodle.
- **Your submission should include all the supplied files** (including demo files) as well as your new files in the zip file you submit.
- **Also include your completed documentation sheet(s) in your submission – as a separate file outside of the zip.**
- If you worked on Mac or Linux, please include your make files etc that you used, ideally in a format which we can use to build and test the code ourselves, along with summary instructions of how we can build and run the program ourselves if necessary.

It should be possible when you have done this for someone to download the zip file, unzip it, do a 'rebuild all' to rebuild it, and run it, with no other changes. (Or for us to build/run it on Linux/Mac if appropriate.)
It should also be possible for us to check your documentation file to see whether the mark matched what you expected or where any issues were.

8. **There are 80 marks available for part 2 requirements**, for a total of 80% of the module mark.
   These are usually a lot harder than the part 1 requirements. **Start with the easy ones! You may set a time limit so that you don't overdo it.**
   Read the requirements carefully!
   When you have completed part 2 upload the project file to the CW2 submission on Moodle, as you did for CW1 (i.e. clean it, delete the .vs directory and zip the folder).
   A demo video is required to show the program running and explain how you meet the criteria.
   You will need to upload a documentation file explaining which features you achieved.

Some clarifications (based on questions in previous years):

The key priority is that you understand the code. The assessment aim is to **test your ability to understand C++ code and to write it** – not to copy paste. Here are some examples of things which are and are not allowed as far as marking is concerned:

- If you take a demo and just add some changes then you are assessed only on your changes.
  i.e. if maze demo already did something (e.g. background drawing, pause facility, etc) then it doesn't count for a mark for you because you did not write it yourself.
  Similarly, it does not count as you creating a new subclass because you didn't.
  Please start from a new class and add your own code to it.
- If instead you write your own code and take from the demos the understanding of how they do it – possibly even copying some parts of the code, *but showing your own understanding of it when you do*, then that is OK. (This is partly why we ask you to demo it – so you can answer questions if necessary.) E.g. if you copy the way that pause works and implement it correctly then you would understand how it works and be able to explain it.
- As another example, if you copy whole functions, such as a draw function, from a demo, that is also a case of something being the same as maze demo. If you do so and make a slight change to make it a different colour then it's basically the same apart from that slight change, so it is not different enough. Don't do that please as you won't get the marks.
- However, you CAN copy small parts into your own classes, showing your understanding of them. In which case you should be able to explain how they work if asked.

General marking criteria for both part 1 (CW1) and part 2 (CW2):

You will lose marks if any of the following apply. In each case we will apply a percentage mark penalty to your mark.

- Your program crashes on exit or has a memory leak. (Lose 10% of your mark.)
- Your program crashes at least once during its operation. (Lose 20% of your mark.)
- Your program crashes multiple times. (Lose 30% of your mark.)
- Your program crashes frequently. (Lose 40% of your mark.)
- Your program has some odd/unexpected behaviour/errors. (Lose at least 10% of your mark.)
- Your program has a lot of unexpected behaviour/errors. (Lose at least 20% of your mark.)
- You have changed framework files without prior agreement. (Lose at least 20% of your mark.)

The mark sheet that will be used for marking parts 1 and 2 has entries for the above and your marker will annotate it according to their experience of your demo (or their running) of your software.

Coursework Part 1 Functional Requirements

Functional requirements: (2 marks each).

Note: the markers will use the **literal text below** (**both the bold and non-bolded text**) to determine whether a requirement was met or not. Please read the criteria carefully to ensure that you don't miss anything. If I need to clarify any of these criteria I will do so and let you know that it was updated.

To try to avoid you missing specific parts I have labelled key aspects of each requirement in a list.

In general, the requirements below get progressively harder (in that they need more understanding of the framework and C++ to do the later ones) but please do check later ones even if you decide some earlier one is too hard for you to do.

1. **Create an appropriate sub-class of BaseEngine with an appropriate background which is different from the demos.**
    a. Create an appropriate new sub-class of the base engine.
    b. Give it an appropriate background.
    c. Ensure that the background is different to the demos.
    d. Name your class using your capitalised username followed by the text Engine. e.g. if your username was psxabc then your class would be called PsxabcEngine.

2. **Show your ability to use the drawing functions:**
    a. Draw your background ensuring that you use at **least one** of the **shape** drawing functions to draw on the background.
    b. and that you draw at **least one image to the background**,
    c. which is different from the demos and shows your understanding.

    A blank background will not get this mark – even if you change the colour – you MUST use one of the shape drawing functions (i.e. a drawBackground…() function other than drawBackgroundPixel()) and at least one image, to show your understanding.

3. **Draw some text on the <u>background</u>.**
    a. Draw some text onto the <u>background</u> (not foreground)
    b. and ensure that the text is not obliterated when moving objects move over it.
    c. Ensure that moving objects can/do move **over** at least part of this text, so that the object appears in front of the text, and demonstrate that it is redrawn after the object has moved.

    i.e. show that you understand how the background and foreground are drawn to and when to draw text.

4. **Have some changing text, refreshing/redrawing appropriately which is drawn to the <u>foreground</u> (not background), in front of moving objects.**
    a. This text may change over time (e.g. showing the current time, or a counter) or could show a score which changes, for example.
    b. It could also be drawn to the foreground as a part of an object (e.g. a moving label) if you wish, but does not need to move around with objects if you don't want it to.
    c. When the text changes, the old text should be appropriately removed from the screen.

d. The text must be drawn such that **moving objects would move under it** rather than on top of it though. i.e. **not to the background**, and basically it means it'll be drawn after at least some of the objects.
e. For marking we will check the code where it is drawn if there is any doubt. E.g. whether the function which draws it is called before or after drawing objects. Look at different functions to see which to use – the point of this mark is to see whether you realised the difference between drawing changing text to foreground rather than background.

5. **Provide a user controlled moving object which is a sub-class of DisplayableObject and different to the demos:**
    a. Have a moving object.
    b. that the user can move around,
    c. using either the keyboard OR the mouse (or both)
    d. and is different to the demos.

    Note: you could have an indirect subclass of DisplayableObject if you wish (i.e. a subclass of a subclass of DisplayableObject). The aim of this requirement is for you to show that you understand how to use EITHER the keyboard or mouse input functions (or both) as well as to show that you can create a moving object.

6. **Ensure that both keyboard and mouse input are handled in some way and do something.** *At least* one of these input methods should influence one or more moving objects, from requirement 5. The starting point is probably to look at whether you used mouse or keyboard for the moving object above and just use the other one here. E.g. if your user-controlled object is mouse controlled then make it so that something happens when a key is pressed (e.g. a counter is incremented, which appears on the screen – see requirement 4). Or if your object is keyboard controlled, you need to handle mouse movement or button pressing. Both mouse and keyboard could influence the moving object if you prefer, or one could change something else.
    a. You handle **both** keyboard input AND mouse input and they both do something.
    b. **Something(s) should visibly change for both** – e.g. some position of something or value of something or displayed image, or…

7. **Provide an automated moving object which is a sub-class of DisplayableObject and different from the one in requirement 5.**
    a. Provide a second moving object (separate to the user-controlled one, with a **different class**)
    b. whose movement is not directly controlled by the player,
    c. which moves around,
    d. which acts differently to the objects in the samples/demos/code that I provided,
    e. and which looks different to the objects in the demos and to your object in requirement 5.

    i.e. show that you understand something about how to make an object move on its own, without user input (changing its x and y coordinates and redrawing it appropriately). Your object must have some behaviour that is different to the demos (i.e. a copy-paste of the demo code is not sufficient) and you must be able to explain how it works and justify that it is different from the demos in some way. Your object must also have a different appearance to the object in requirement 5 and look different to the demos/samples as well.

8. **Include collision detection between a user-controlled (req. 5) and an automated (req. 7) moving object, so that they interact with each other.**
    a. You need to check for a collision between the two objects.
    b. Something should happen when they collide, and something should visibly change – e.g. something moves, direction of travel changes, or something is shown.
    c. Collision detection should be at least as good as rectangle-rectangle interaction and should work properly.

    This means that at least two of your objects should react to each other when they collide. Hint: look at UtilCollisionDetection.h if you don't want to do the maths for rectangle or circle intersection yourself – and see MazeDemoObject.cpp for an example of using these functions. Assessment of whether you achieved this or not will be on the basis that the intersection of two objects is correctly assessed and something happens in reaction to this (e.g. objects move, change direction, something else changes (e.g. a score) etc). Rectangle-rectangle or circle-circle interaction is fine for meeting this requirement.
9. **Create your own subclass of TileManager**.
    a. Create a subclass of the tile manager which has different behaviour (at least a little) to the demos, is drawn to the background, and is visible to the user when the program is run.
    b. Name your class using your username followed by the text TileManager. e.g. if your username was psxabc then your class would be called PsxabcTileManager.
    c. Explain the difference(s) from the demo versions.

    Hint: Look at the existing demos, including the bouncing ball demo. Display the tile manager on the background, so that the tiles are visible. It must be different from the demos but can still be simple. You are just showing that you understand how to use the tile manager class. Use a different tile size and number of rows and columns to the demos (e.g. 13x13 tiles and 6 rows and 9 columns if you can't think of some numbers yourself). Your TileManager must not be a copy of an existing one, or just an implementation of the demo tutorial example without change. i.e. you must show some understanding of how to do this, not just blindly repeat the steps of demo tutorial A.

10. **Have at least one moving object interacted correctly with the tile manager, changing a tile:**
    a. Ensure that at least one of your moving objects visibly changes specific tiles when it passes over them – using the position checking appropriately.
    b. The tile must be changed and redrawn correctly so that the user sees the change.

    Consider the bouncing ball demo if you can't work out how to do this from the information in demos A and B, but you need to have at least some difference in behaviour from that. Assessment will check that you correctly detected the specific tile that the moving object was over and handled it appropriately.

Coursework part 2 functional requirements

Again, key parts of requirements have been highlighted with letters to try to ensure that you don't miss something – you need to meet ALL the lettered sub-requirements to get the mark.

The following requirements apply, and have a variety of marks available:

Handling of program states (total max 6 marks, 2 are advanced)

1. **Add states to your program (max 6 marks).** This means that your program correctly changes its behaviour depending upon what state it is in. Each stage should have a correctly drawn background which is different to the demos, and are **NOT** trivial (e.g. a blank background). This could use an image, a tile manager or be drawn using the fundamental drawing functions on the base engine. You can get a variety of marks for this:

   2 marks:
   a. a start-up state, a pause state and a running state,
   b. which differ in some way in **both** the appearance and behaviour.

   2 marks:
   c. at least five states
   d. including at least one win or lose state as well as those mentioned above. (Note: if you are not doing a game, then have a state which allows a reset, e.g. 'load new document' in a text editor.)
   e. There must be significant differences between the states in behaviour and/or appearance. The program must be able to correctly go back from the win/lost (or reset) state to the starting state to start a new game/document, correctly initialising any data.

   2 marks (advanced):
   f. implement the state model (design pattern) using subtype polymorphism rather than if/switch statements in each function. Note: You may still need some if statements etc to enter the states, but the current state object is used to determine how to act in each function.

   Look up the 'State Pattern' to see what this means and think about it. These are advanced marks, so we will not explain how to do this beyond the following: there will be a basic state base class and a subclass for each of the different states. Your BaseEngine sub-class will need to know which state object is currently valid and the different methods will call virtual methods on this object. The different behaviour will therefore occur due to having a different object being used for each state rather than having a switch in each of the methods. If you have if or switch statements specifying what to do in different states, then you won't have done it properly, so you won't get these marks. You should NOT have more than the one sub-class of BaseEngine if you do it correctly. If you had to create multiple sub-classes of BaseEngine then your implementation is wrong (and there will be other issues since you may have more than one window as well).

Input/output features (total max 6 marks, 2 are advanced)

2. **Save and load some non-trivial data (max 6 marks).** You can use either C++ classes or the C functions to do this – your choice will not affect your mark.
   You can get a variety of marks for this:

   2 marks: most basic saving and loading, as follows:

a. save AND load at least one value to/from a file, e.g. a high score to a file (e.g. a single number to a file)

2 marks: completed the saving/loading above, and **load some more complex, structured data**, e.g. map data for levels, or formatted text documents where the formatting will be interpreted. The main criteria are:

b. It must be multiple read/write operations, of multiple values with some kind of structure to the file, rather than just a string of 3 numbers, for example.
c. Something must visibly change depending upon what is loaded (e.g. set tiles according to data read and/or set positions of moving objects according to the data).

2 marks (advanced): completed the above but also save/load the non-trivial state of the program to a file.

d. A user should be able to save where they are (e.g. the current document that they are working on or the state of a game – saving **all** relevant variables for **all** objects)
e. and they should be able to **reload this state later**, with everything acting correctly and continuing from where it was.
f. Note: this means saving/reloading the positions/states of all moving objects as well as anything like changeable tiles, etc.
g. You will need to provide some way to reload from the state as well – e.g. when the program starts (e.g. choosing to load or start a new game) or in response to some command from the user (e.g. pressing S for save and L for load?).

This is meant to be non-trivial and may need some thought and debugging to make it work properly.

Displayable object features (total max 15 marks, 6 are advanced)

3. **Use appropriate sub-classing with automated objects (max 2 marks):**
   a. multiple displayable objects
   b. from at least three displayable object classes,
   c. with different appearances and behaviour from each other
   d. and you should have *an intermediate class* of your own between the framework class and your end class,
   e. the intermediate class *adds some non-trivial behaviour*.

   i.e. you are showing that you can create a subclass which adds some behaviour, and some other subclasses of that class which add more behaviour.

   Example: a complex example can be found in ExampleDragableObjects.h:
   DragableObject and DragableImageObject are both DragableBaseClassObjects, which is a DisplayableObject, so DragableBaseClassObjects is an intermediate class which adds some non-trivial behaviour.

4. **Create and destroy displayable objects during operation of a state (max 4 marks):** for example, add an object which appears and moves for a while for the player to chase if a certain event happens, a bomb that can be dropped by a player, or a bullet that can be fired. Note that something like pressing a key to drop a bomb which then blows up later, while displaying a countdown on the bomb, and then changes the tiles in a tile manager would meet some requirements (i.e. marks) in one feature.

2 marks: Objects appear to be created or destroyed over time, which means you:

a. dynamically add one or more displayable objects to the program temporarily after a state has started (i.e. you add to the object list *or* make some object visible).
b. and ensure that the object cannot interact with anything else before it is added or after it is 'destroyed'. E.g. if you make the object invisible then it is not collision detecting with anything while it is not visible.
c. You must not just re-create the entire object array contents to do this, although you could add to the end of it. (Please see the methods on DisplayableObjectContainer.)
d. This requirement is not met by the re-creation of objects when you change states. The idea behind this requirement is that moving objects appear and disappear while using the program within the state.

For the above criterion you can hide/show an object rather than actually creating/deleting them. Remember if you do though that you may also need to ensure that their own and other DoUpdate() functions ignore the objects. e.g. just because you don't draw an enemy on the screen, the collision detection may still check for it if you don't stop it doing so. If it is unclear, we may ask you to show us in your code how you avoid this problem.

2 marks: as for the above 2 marks but also correctly create and destroy displayable objects during operation**.** This means meeting the requirements above but also:

e. when making the object appear during the running of one state, create the object and add it to the DisplayableObjectContainer (rather than just making it visible),
f. when making the object disappear, within the running of the state, remove it from the object list in the DisplayableObjectContainer.
g. notify the program when you change the contents of the DisplayableObjectContainer. If you change the object array you need to tell anything using it that you did so, hence why this is in as a separate requirement – it can be trickier to get right. Please see the drawableObjectsChanged() method and investigate how it works to get this right.
h. correctly delete the object at that time (not at some later point, such as when the state changes). Making sure that you destroy/delete the object at the right time is important for this – it is trying to get you to understand how to get objects to destroy themselves safely without causing issues with using the pointer after the object was destroyed. This is an important thing to understand in C++.

5. **Complex intelligence on an automated moving object (max 9 marks):** this is a harder criterion to assess since there are so many different ways you could do this. As a minimum this criterion would involve something more than moving randomly or homing in on a player, but you should consider the marking criteria below to work out what sort of mark you will get.

2 marks: this could involve something like 'If the player is on the same column, then home in. Otherwise move randomly', or 'Keep the same direction until I bump into something then change direction towards player and repeat'.
Simple equations and decisions would get this mark rather than a higher mark.
Note that this must be more than a single constant behaviour – e.g. the moving randomly or homing in behaviour, so that **the behaviour changes somehow according to the situation**.
If your logic can be formulated as "**if <condition> then do behaviour type 1, else do behaviour type 2**" where behaviour types 1 and 2 are things which themselves involve a decision.
e.g.:

- "If player is within 200 pixels, go towards player, else move randomly".
- "If in this area of the screen, move using these rules, else use these rules…"

2 marks: this means a good implementation of the intelligence of a moving object.

There should be some level of calculation or multiple-decision-making element involved.
This needs to be more complex than the previous two-mark criteria (i.e. it could not be expressed in the way that the two-mark criteria could) and could involve some more complex calculations or a series of decisions.
E.g.:

- calculating how to cut off a player, or intercept them, assuming that they maintain the same speed (predict where they will be and plan a path to get there)
- Showing some apparent intelligence which is not obvious to the markers how to implement, and not just random.

Note that the important thing for marking here is the skill you **show in your C++ ability** to write algorithms by implementing this – so something trivial will not count.

2 marks (advanced): this means a **good implementation** of some **more complex algorithm** for the intelligence of a moving object,
e.g.:

- use a shortest path algorithm to find the shortest way through a maze to get to a player,
- tracking a player's decisions and predicting a player's path (in some non-trivial way, such as understanding what player decisions have been so far and what they may imply for later behaviour) and moving towards that rather than the player itself.
- showing some apparent intelligence, so that the markers are impressed at how intelligent the object(s) appear to be.

Note that the important thing for marking here is the skill you show in your C++ ability by implementing this – so something trivial will not count.

3 marks (advanced): this is an even more advanced version of the above criteria. This means:

- An **exceptional** implementation
- where the marker **is impressed with the complexity of the problem** you are solving,
- and the elegance of the C++ code that you are using to implement it.

Note that this talks about the complexity of the problem, rather than an overly complex algorithm. This is deliberately not easy to get and is designed to allow the most capable students to excel. For the most capable students, doing things like this will not take as long as many of us may think. For the rest of us, it is probably too time-consuming to be worth the time to do it.
Please don't be upset if you don't get this mark.
One key criterion to consider is:

- "Does this show you to be one of the best C++ programmers in the year?"

Collision detection (4 marks, 2 is advanced)

6. **Non-trivial pixel-perfect collision detection between objects (max 4 marks):** this means to implement collision detection beyond the collision detection that I gave you examples of. These marks are for implementing some really complex collision detection, such as complex outline interactions (e.g. someone did bitmap-bitmap interaction in the past, checking for coloured pixels interacting, and someone else split complex shapes into triangles which could be collision detected and checked every triangle interaction) then you get this mark. Using the supplied collision detection class is not sufficient to get either mark. Collision detection for rectangles and/or ovals is not sufficient.

   2 marks: improved collision detection which will work on more complex shapes than the supplied collision detection methods, and which works well in your program. Particularly that will work with convex shapes. E.g.:

- Putting a more convex polygon around the shape and accurately detecting collisions, or having collision detections between a range of polygonal shapes (triangles, squares, pentagons, etc), which goes beyond the square-square or circle-circle collision detection. As implied, this is likely to be most appropriate if your shapes are already polygons.

2 marks (advanced): pixel-perfect collision detection on an **arbitrarily complex irregular shape**. Importantly, this should include dealing with concave shapes, so that one could be within a concavity of another but not count as a collision. (Think of a circle being able to move inside a C shape, and as long as it doesn't touch the C then it is not a collision.)
Note that this is hard to get, so if your implementation is not solving a really complex task and/or was trivial to code then you probably will not get this mark.
Your method should work with concavities in object outline and your demo should show this.
Examples would be **automatic accurate** triangularisation of shapes and collision detection on these, or pixel-perfect checking (if a coloured pixel in one image is in the same place as a coloured pixel in another image).
Although it may not seem like it, the easiest way to get a good implementation of this requirement is to use images and to check every pixel in the overlap area to see whether the relevant position in both images is coloured. (If either is not coloured then there is no collision at that point.) If there is a collision for ANY pixel in the overlap area, then the objects have collided.
In particular, for 2 marks:

- Your approach should work even if the shapes of the objects colliding are modified (potentially with some minor changes to account for shape types or colours). i.e. it should use the object itself, not some hard-coded representation of the object, for instance of coordinates of the edges.
- Your method should work with concavities in objects outlines and your demo should show this. (Note: checking for pixel collisions in images will do this automatically.)

Animations, foreground scrolling and zooming (Max total 15 marks, 3 are advanced)

7. **Implement a scrolling background by manipulating the way that the background image is drawn (2 marks).** This requirement basically requires you to look and understand the StarfieldDemo example, which manipulates the position at which the background is drawn.
   You do not need to have a constantly scrolling background – it could scroll in response to some activity, such as the player moving for example.
   You may want to implement this for a startup screen, high score screen or ending screen if it does not really fit with your main screen display, or to think about how it could be incorporated into your program.
   Using a background surface which is bigger than the displayable screen could give the illusion of having a view onto a larger background image, and moving around it.
   The main requirements are:
   a. Setup an appropriate background image to use (which may be bigger than the window).
   b. Modify copyAllBackgroundBuffer() appropriately to work with your background and ensure that it is redrawn as needed (redrawDisplay()).
   c. Demonstrate an apparent smooth scrolling of the background.
   d. Explain how you did it, showing your understanding.

   The aim is for you to work out for yourself by looking at the example code, so we will NOT explain to you how the code works – please don't ask.

8. **Have an animated or changing background by utilising multiple images (2 marks).** Having multiple drawing surfaces is an important concept which is widely used, and is easy to do using pointers. This requirement basically asks you to consider and understand the FlashingDemo sample. You need to:

a. Have at least five drawing surfaces, set up with at least slightly different contents.
b. switch the appropriate surface in to be used as the background (noting that m_pBackgroundSurface is accessible to subclasses and can be changed, deliberately for this purpose).
c. Make it appear as if the background has been animated (at least slightly) using this process. E.g. students in the past made torches flicker, things change shape, etc. The changes should look smooth rather than the (deliberate) big changes used in the demo.
d. Explain how you did it, showing your understanding.

The aim is for you to work out for yourself by looking at the example code, so we will NOT explain to you how the code works – please don't ask.

9. **Correctly implement scrolling and zooming of the foreground, allowing the user to scroll around using keys and/or mouse (max 4 marks).** Hint: look at the FilterPoints class to see how this can be done relatively easily, and consider the ScrollingAndZooming demo. You do NOT need to scroll or zoom the background (although you could integrate this with requirements 8 and 9 if you wished to do so).
   One aim for these marks is for you to work out for yourself how to do this, or to work out from looking at the code which uses FilterPoints, so we will NOT explain to you how the existing code works – please don't ask.

   2 marks: you wrote C++ code which works to:
   a. allow a user to scroll the screen.
   b. using keys or mouse.
   c. The user must be able to move the apparent view on the foreground objects up, down, left AND right (you can choose which keys/mouse operations do this).
   d. Note: there is no requirement to actually use the FilterPoints class to do this for this 2-mark criterion, although it will be simple to do if you already plan to do the next 2-mark criterion. So, in theory you could manipulate the positions of all the objects manually if you wanted to, as long as it works properly, if you can't understand FilterPoints.

   2 marks: you achieved the above but also:
   e. implemented **both** scrolling and zooming of foreground objects.
   f. using the supplied FilterPoints class by creating a FilterPoints subclass which works to allow a user to zoom in and out using keys or mouse.
   g. Your code should show your understanding of how to integrate your zooming with the framework provided – using the FilterPoints class that you create, and explain how it works in the documentation/demo video.

10. **Animate moving objects (max 5 marks):** To get these marks you need to show your ability to create smooth animations for moving objects. Note: there are various advanced demos which show you how you could do some elements of this, but you should not just switch between two images (as one of the demos does), it should look smooth (e.g. having enough images to be smooth, or looking at how to make it smoothly animate in another way).
    This cannot just be a copy-paste of the demo code – even for one mark you need to show enough awareness of what you are doing to justify that you 'showed understanding'.

    2 marks: for this you need to:

a. show understanding of how to animate at least one object so that its appearance changes over time, however this could be a proof of concept instead of being smooth.

3 marks (advanced):

b. your objects should have animated rather than fixed appearances.
c. the animation should be *smooth.*
d. **and** *visually impressive*. Please do note the 'impressive'. So please don't just rotate objects using facilities from a demo as that would not get you the marks.
e. Explain in the documentation/demo what you have done and how it works.

11. **Image rotation/manipulation using the ImagePixelMapping object (max 2 mark):** you need to show that you understand how to create a new ImagePixelMapping class which acts differently to the existing ones and does something at least slightly differently to the existing examples, and that you use it appropriately.

    2 marks: show your understanding by:

    a. creating and using your own ImagePixelMapping class and object
    b. draw at least one object to the screen. Your class must do something different enough to the provided ones to allow the marker to see that you understand how to use it fully.

Tile manager usage (Max 4 marks, 2 are advanced)

12. **Interesting and impressive tile manager usage (max 4 marks):** To get this mark you must be using a tile manager, and must have multiple displayable objects.
    - Your tile manager must draw a number of appropriate and different *pictures* (either using images or the drawing primitives) for the different tile types, which are not just different colours.
    - You should have at least 5 different tile types (not just different sizes ovals/rectangles).
    - At least one tile must be drawn using an image. You should load the image only once and keep it in a relevant object, NOT keep reloading it each frame if you want these marks.

    Note: If you use multiple images then you could also potentially meet the animated appearance of automated objects criterion, and/or the animated background if you do it appropriately.
    2 marks: You meet the wording above in all requirements.
    2 marks (advanced): The marker is impressed by what you have done, it looks great, works well and has some interesting behaviour. Also, you particularly need a situation in which moving onto some tiles would have effects on tiles elsewhere, e.g. standing on a key tile visibly unlocks a door tile. This requires *visibly* changing one tile from a position where the displayable object which does so is NOT over that tile (i.e. you update a different part of the screen.)

Other features (Max total 10 marks, 6 are advanced)

13. **Allow user to enter text which appears on the graphical display (max 2 marks):** For example, when entering a name for a high score table, capture the letter key presses and pressing of delete key, and show the current string on the screen (implementing delete at well may be important). This needs thought but is useful to demonstrate your understanding of strings. I added this optional requirement because some people did it anyway last year for entering high score tables and I wanted the marking scheme to reflect that it was done by giving a mark for it. Entering text into the console window does not count for this.

    2 marks: Do the above, including meeting the following key requirements:

    - Capture the key presses for letters/characters.
    - Store the key presses somewhere.

- Capture the DELETE key press and handle it appropriately.
- Display the text on the screen.

14. **Show your understanding of templates, operator overloading OR smart pointers (max 2 marks).** To get this mark you should use at least one of these features in an appropriate and useful manner in your program. Using one of these is sufficient.

    2 marks: for this mark you need to:
    a. have created and used a template class, template function, operator overload (other than = and ==) or smart pointer
    b. in a way which shows your understanding of how to use it
    c. and is different to any provided examples/demos.
    d. and uses it in an appropriate and non-trivial manner.
    e. to achieve something useful, for which it is the appropriate feature to use.

    Hints:
    - You may want to consider whether you can use operator overloading as a part of your loading/saving, since you can use global functions to overload << and >>.
    - If you use normal pointers, it may be an easy change to convert them to smart pointers once you understand what these are.
    - If you have something which can be applied to multiple classes, which cannot be resolved using sub-classing then consider whether a template is appropriate.

15. **Additional complexity (max 6 marks, advanced).** The students who find C++ easiest often have great ideas for things that they would like to do, which display good knowledge of C++, are complex and interesting but don't fit into these marking criteria. This mark is available for covering features where you show a good knowledge of C++, use some complexity, or create something that is exceptionally impressive. The mark aims to reward only the students who are the very best at C++ and is an advanced mark.
    Explain in your documentation (demo video) if you think that you are doing something extremely impressive in some way which is not covered by the marking scheme as written.

- Importantly it has to show exceptional ability in C++, and must cover some feature which is not covered by another criterion.
- It may be possible to get these marks for a topic covered by one of the other marking criteria if your implementation was truly impressive, well beyond the scope of the existing mark scheme.
- It is also possible to get the mark for something which demonstrates a useful extension to the framework which shows your significant understanding of C++.
  Important: You need to specify clearly what have done, why it is beyond the current criteria and how it demonstrates your considerable skill with C++!

Overall impact/impression (Max 20 marks, 20 are advanced)

16. **Impact/impression/WOW factor! (max 20 marks)** These marks are awarded based on the overall impact/impression of the program, taking into account its comparative standing and uniqueness within the cohort. The assessment considers whether the program distinguishes itself in a way that would motivate people to pay for it, relative to other programs in the group. These marks will take some effort to attain and are designed to ensure that the people who are most capable with C++

get higher marks. Be careful not to spend too long on this coursework if you are struggling! Many people will just do the minimum and will not get these marks (e.g., your program works well but only just meets the requirements rather than having a great impact or impression). These are meant to be advanced marks, so don't be disappointed with this.

5 marks (advanced): This is non-trivial to get. You made an effort to ensure that it is beyond the minimum to just tick boxes.
As a minimum this means:

- you made some effort with the graphical appearance.
- the background is relevant and not plain or the same as any of the demos.
- the background includes at least some use of relevant shapes (e.g. separating off a score by putting it in a box and labelling it) and/or images.
- moving objects are not just plain circles or squares.
- have at least three moving objects.
- at least one is user controlled.
- you accept both mouse and keyboard input in some way.
- you use both images and drawing primitives appropriately.
- you have appropriate states (at least 2).
- have something beyond the minimum required for the other marks (*please say what*).
- your program should work smoothly.
- everything looks good with no problems.

You need to include one or more screen shots in your documentation, along with an explanation of what you did that was beyond the minimum in order to get these marks.

5 marks (advanced): Meets all above criteria and it is also *very impressive for the markers*. It needs to work very well, doing some complex tasks and you obviously made a significant effort on this coursework. If it's a game then it's fun and interesting to play, and has at least 2 different levels (to demonstrate your ability to do this). As a non-game (e.g. a drawing package or word processor?) it should be a complex program with different states (e.g. different page view layouts) performing a useful task and it should achieve its purpose well.
It should be impressing the markers to get these marks.
You need to provide information in your documentation about why this is so impressive, and include some screen shots, making an argument for getting these marks.

10 marks (advanced): This is supposed to be even harder to get and basically means that your 'impact' of your program was even more than the above mentioned impact/impression, which will not be easy. This basically means that all the markers went 'wow' when they saw this and thought that people would easily pay money to buy this program.
These marks are there so that we can differentiate the quality of coursework. In the same way that one would not always expect to be able to get 100% in an exam, not all students would be able to achieve this mark in a reasonable time, so should not be trying to do so.

Note: this mark is to assess the C++ ability, not the length of time in level design or data entry. As long as we can see where this is going, you should not spend too much time designing lots of extra levels or entering a lot of data for use in the program. You shouldn't need to be designing more than three levels as a proof of concept, if the levels you design illustrate your program well.

Your program should be working really well, smoothly, look good and make a good demonstration and should be comparable in quality to the sort of programs available on an app store.
I also note that in some cases it is possible that a program could be sellable while not implementing too many of the other features, since some app store programs are relatively simple but still sellable if presented well.
To get these marks you need to provide the same level of documentation as for the 10 marks, but also a short introduction in the demo video of it running and the sort of sales pitch that one would see in an app store, to illustrate why it is so great and why people should buy it.

**** For all these marks you need to include in your submitted documentation a clear description of what you did and how you did it (justifications may also be required), including any screen shots or diagrams, and upload a video demonstrating it working.***